Topology-Aware Spanning Tree Initialization for the Edge-Based Quantum Approximate Optimization Algorithm

Ashar Usmani
Department of Computing
FAST NUCES
Karachi, Pakistan
ashar.usmani9@gmail.com

Minhaj Mateen
Department of Computing
FAST NUCES
Karachi, Pakistan
k224519@nu.edu.pk

***Abstract*— Quantum computing has emerged as a promising field to tackle problems which are computationally expensive and resource intensive on classical computers. In this study, we explore an alternate method to generate Spanning Tree for Edge-based quantum approximate optimization algorithm (QAOA) to the MAX-CUT problem which involves a topology-aware algorithm. MAX-CUT aims to partition the vertices of a graph into two subsets such that the number of edges between the subsets is maximized. We define the naive method for Spanning Tree generation and the topology-aware alternate method for generating Spanning Tree to apply QAOA specifically tailored to this formulation.**

Keywords—MAX-CUT problem · Quantum algorithm · Quantum approximate optimization algorithm · Noisy intermediate-scale quantum

I. Introduction

In the current era of quantum computing, noisy intermediate-scale quantum (NISQ) devices are heavily researched. NISQ computers represent the current generation of quantum hardware, featuring tens to over a thousand qubits, but they suffer from significant noise and errors which limit their capabilities. When designing any algorithm for these machines, these physical limitations must always be kept in mind. Variational quantum algorithms (VQAs) are one of the main branches of algorithms designed specifically to run on NISQ devices [1]. Among the algorithms in this family, the variational quantum eigensolver (VQE) is often used for estimating the ground-state eigenvalues and eigenvectors of a given Hamiltonian [2]. Similarly, the quantum approximate optimization algorithm (QAOA) is known for approximately solving complex combinatorial problems [3].

QAOA is essentially a hybrid algorithm which leverages both quantum and classical computing. The main component is a parameterized quantum circuit that can be tuned. By running the circuit and measuring the output, we can estimate how close we are to the optimal answer. A classical computer is then used in a loop to calculate and adjust these parameters for the next run. The MAX-CUT problem is a classic use case that can be formulated as a special case of QAOA, where the objective is translated into finding the ground state of the problem Hamiltonian.

Traditionally, solving MAX-CUT with QAOA involved assigning 1 qubit to each vertex of the graph. However, recent improvements changed the assignment from the vertices to the edges [4]. The base paper we used for our research builds on this edge-based approach by generating a spanning tree to map the graph's connections. The issue with their methodology is that the spanning tree generation was very rigid, creating a generic structure (like a star graph) without really knowing anything about the actual topology of the graph being evaluated. Because this method ignores the natural structure of the data, it forces unnatural paths and creates unnecessary quantum gate overhead. In this paper, we propose a topology-aware spanning tree optimization method prior to circuit initialization, demonstrating how classical graph search techniques can significantly reduce the CNOT gate count and overall circuit depth.

II. Methodology

The main goal is to find a way to make edge-based QAOA more efficient by changing how the initial quantum circuit is set up. In this section, we'll explain the step-by-step process we used to move away from the basic Star Graph approach and instead use a Topology-Aware spanning tree. We started by looking at the mathematical relationship between the spanning tree structure and the CNOT gate count, since that's the biggest bottleneck for NISQ devices. From there, we developed a classical pre-processing algorithm that searches for an optimal tree before the quantum part even starts.

A. Qubit Mapping

In traditional QAOA each vertex of a graph is usually assigned one qubit. But for our research, we follow the edge-based mapping from the base paper. This means we only need *N-1* qubits for a graph with *N* vertices. Basically, we first find a spanning tree G_e and map each edge of that tree to a qubit. The whole idea is that this tree acts as a skeleton for the rest of the graph. Any edge that isn't in the tree has to be represented by a path through the tree edges, which is where the gate overhead comes from.

B. Star-Graph Baseline

The paper we are building on used a very specific way to make this tree. They just look for the node with the highest degree and connect it to everything else to make a "Star Graph". This is a greedy approach and it's very fast, but it doesn't really consider the graph's actual shape. In a star graph, the distance between any two nodes is at most 2. While that sounds good, it forces a lot of fake connections for edges that are actually far away from the center hub.

This creates a baseline CNOT cost that we think can be improved if we actually look at the graph topology.

C. CNOT Cost Function

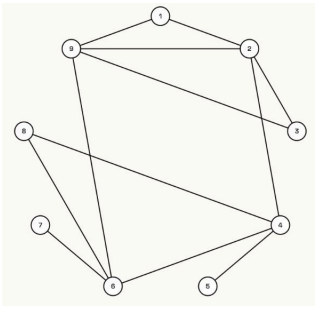
To actually measure how good or bad a spanning tree is for our circuit, we use the CNOT cost formula. Given the graph $G = (V, E)$, let V be the set of n vertices and E be the set of edges. Then $(j, k) \in E$ where j and k are elements of V and l_{jk} be the distance between the elements in our spanning tree G_e. The formula for the gate cost is:

$$N(G_e) = \sum_{(j,k) \in E} 2(l_{jk} - 1) \quad (1)$$

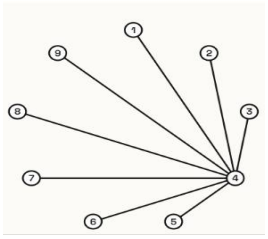
If an edge is already in our tree then the cost is 0 because $l_{jk} = 1$. If the path is longer, the CNOT count goes up really fast. Our goal is to find a tree that makes this total sum as small as possible.

D. Proposed Topology Construction and Algorithm

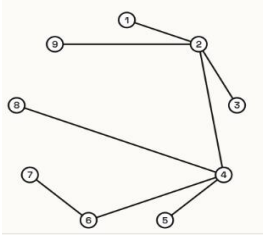
Instead of using a star graph, we propose a classical optimization step before the quantum circuit starts where we use a local search algorithm. First, we start with a basic BFS tree from the max-degree node. Then, the algorithm starts looking at edges in the original graph that are not in the tree. We try adding one of those edges, which creates a cycle. We then test every single edge in that cycle to see which one we can remove to get the best drop in the total CNOT cost. We keep swapping edges like this until we can't find any more swaps that reduce the cost. This way we find a topology-aware tree that might have some long paths but keeps the most important, high-density areas of the graph at a path length of 1. Algorithm 1 summarizes the approach.



(a)



(b)



(c)

Fig.1 Illustration of nine-vertex graph and its corresponding spanning trees. **a** The original graph. **b** Star tree generation mentioned in base paper. **c** Topology aware generation.

Algorithm 1 Topology-Aware Spanning Tree Optimization

1. Identify the root node as the vertex in the input graph G with the highest degree.
2. Construct an initial undirected spanning tree G_e using a Breadth-First Search starting from the root.
3. Identify the set of non-tree edges that exist in the original graph but are not in the current tree G_e.
4. For each non-tree edge, find the unique fundamental cycle it forms with the existing paths in G_e.
5. Perform a trial swap by adding the non-tree edge and removing an edge from the cycle, then evaluate the resulting CNOT cost.
6. If a swap reduces the total cost, update the tree structure and repeat the search until no further reduction is possible.

After the spanning tree is initialized, the rest of the steps are summarized by Algorithm 2. We have not discussed any of the algorithm's steps in detail because they do not help us in any way for comparing the two initialization approaches.

Algorithm 2 Quantum Approximate Optimization Algorithm for Edge-Based MAX-CUT Problem

7. Prepare $n - 1$ qubits in the initial state $|+\rangle^{\otimes(n-1)}$ where each qubit is in $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.
8. For all qubits, perform the problem Hamiltonian H_P, and the mixing Hamiltonian H_M, operations with angles γ and β.
9. Repeat steps 2-3 for p levels, allowing different angle parameters in each iteration.
10. Measure the final quantum state and compute the cost from the obtained measurement results.
11. Optimize the angle parameters to maximize the computed cost.

III. Results

In this section, we present simulation results for various random graphs. All random graphs were generated using Python's NetworkX library and only connected graphs were selected.We used IBM Qiskit's qasm_simulator for these simulations. We employed the constrained optimization by linear approximation optimizer.

For random graphs with 15 and 20 vertices and 100 or 150 edges, we conducted simulations for the two types of spanning tree generation: star based and topology aware. The results are presented in Fig. 2 where the horizontal axis represents the QAOA level (up to level 3) and the vertical axis indicates the approximation ratio. The red line corresponds to star-based generation and the blue line corresponds to topology-aware generation. Solid lines indicate results for graphs with 15 vertices, while dashed lines represent graphs with 20 vertices. Circle markers denote graphs with 100 edges, and triangle markers denote those with 150 edges. For each graph configuration, we randomly initialized the QAOA angle parameters. We simulated each configuration using 10 independently generated random graphs, and 100 different initial angle

parameter sets were applied.Across all QAOA levels, the two generations demonstrate nearly identical approximation ratios where topology-aware outperforms star-based in some cases. This indicates that topology-aware generation gives identical results with less complex circuit and less utilization of resources.

The circuit depth results for random graphs with 15 vertices are present in Fig. 3. We conducted simulations by fixing the number of vertices to 15 and varying the number of edges as 60, 80, 100, 120, and 140 to generate random graphs. For each case, 1000 random graphs were generated by using different random seeds. The methods employed to generate spanning trees were star-based generation and topology-aware generation. Both methods were run with the same seed for all the 1000 graphs.

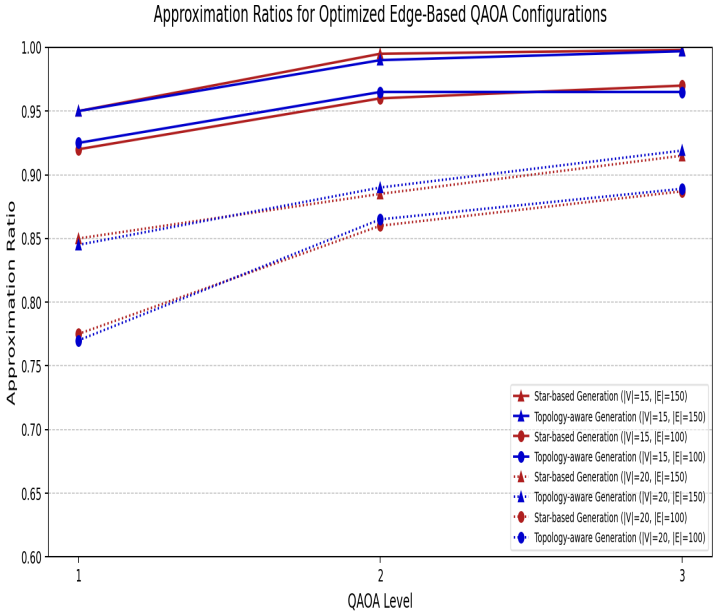


Fig. 2 Approximation Ratios for Different Random Graph Configurations. The figure presents simulation results for star-shaped initialization and topology-aware initialization. The vertical axis represents Approximation Ratio(algorithm answer divided by the actual answer), while the horizontal axis represents different QAOA depth level. Star-based generations are denoted by red lines while the topology-aware generations are denoted by the blue lines.

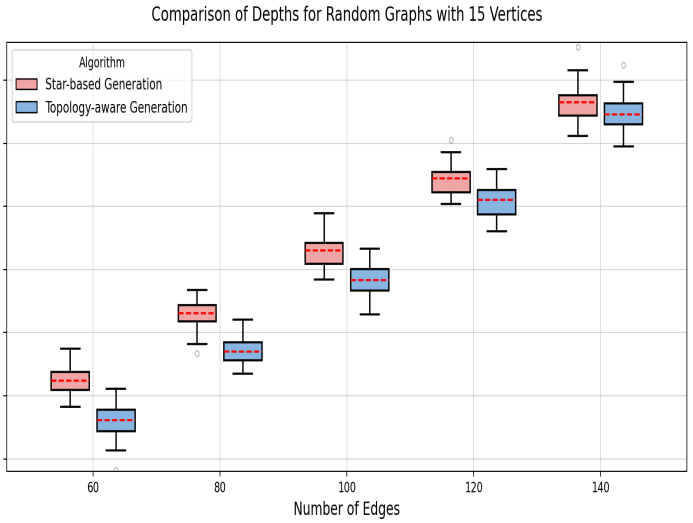


Fig 3. Comparison of quantum circuit depths for random graphs with 15 vertices across counts (60, 80, 100, 120 and 140 edges). The vertical axis represents the quantum circuit depth, while the horizontal axis denotes different graph cases. Star based generations are denoted by the red boxes while the topology based generation are denoted by the blue boxes. The median is represented by the dotted red lines in the boxes.

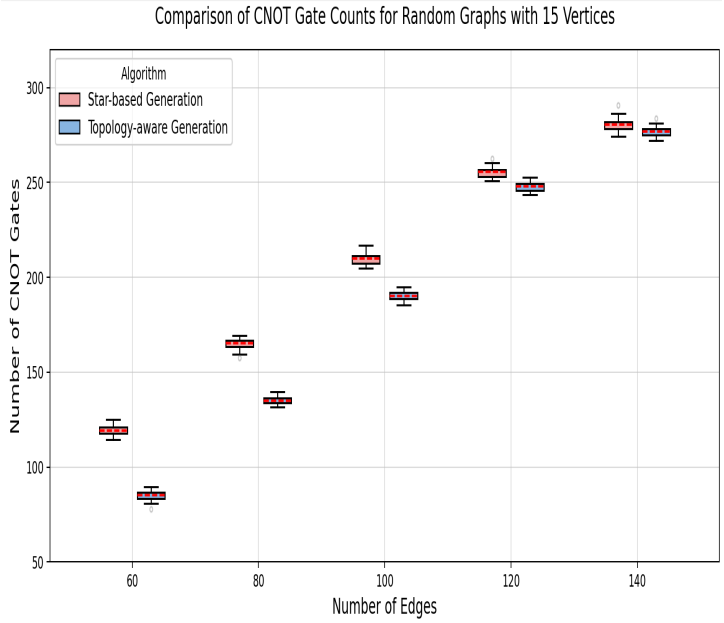


Fig. 4 Comparison of number of CNOT gates for random graphs with 15 vertices across counts(60, 80, 100, 120, and 140 edges). The vertical axis represents the number of CNOT gates required in the circuit while the horizontal axis denotes different graph cases. Star based generations are denoted by the red boxes while the topology based generations are denoted by the blue boxes.

Fig. 4 presents the comparison of CNOT gate counts in quantum circuits. The vertical axis represents the number of CNOT gates, and the horizontal axis distinguishes different spanning tree initialization cases with 15 vertices fixed. For both methods, the box plot shows little variance. The results indicate that the topology-aware initialization consistently reduces the number of CNOT gates across all cases.Furthermore, in higher-density graphs, the difference becomes lower as the number of paths with $l_{jk} = 1$ increase.

In Figs. 3, 4 complexity simulations were performed for various random graphs with different edge counts and densities. Summarizing the simulation results, the topology-aware initialization exhibited lower quantum circuit depths and CNOT gate counts, especially for lower density graphs. As graph density increased, the differences in circuit depth and CNOT gate counts reduced as the topology-aware method also leans towards a star graph for higher densities due to more centric structure of the graph. This may be the reason why the circuit depth and CNOT count for complete graphs are exactly equal.

IV. Conclusion

In this study, we introduced a topology-aware spanning tree initialization method designed to optimize edge-based QAOA for the MAX-CUT problem. By replacing the rigid star-graph baseline with a classical local search algorithm that minimizes the CNOT cost function mentioned in (1), we demonstrated a reduction in the quantum resources required for circuit execution. Our results indicate that this classical pre-processing step successfully identifies an underlying skeleton for the graph that minimizes the paths required to represent non-tree edges, and lowers the overhead associated with edge-to-qubit mapping.

Crucially, the simulation results confirm that the reduction in circuit complexity does not come at the cost of algorithmic performance. Across various graph configurations and QAOA levels, the topology-aware approach maintained approximation ratios nearly identical to the star-based baseline. This showcases our method as a good alternative for NISQ devices, where minimizing CNOT gate counts and circuit depth is vital for reducing decoherence and gate errors. The ability to achieve parity in solution quality with fewer gates represents a meaningful step toward making edge-based VQAs more practical for current hardware.

Finally, our analysis of graph density provides important insights into scalability of spanning tree-based initializations. While the topology- aware method offers a significant reduction in low-to-medium density graphs for circuit depths, we observed a convergence in CNOT counts and depths as graph density increases towards a complete graph. This behavior suggests that as the graph's centricity increases, the optimal spanning tree naturally approaches the star-graph configuration. Future research could focus on further refining these classical search techniques or exploring hybrid mapping strategies to maintain these efficiency gains across even broader classes of combinatorial optimization problems.

Acknowledgment

We are deeply grateful to Yongjin Seo and Jun Heo for their pioneering work, which served as the mathematical cornerstone for our study. We also wish to acknowledge Ms. Sumaiyah Zahid for her dedication and support in teaching us the principles of Quantum Computing, providing the necessary expertise to carry out this experimentation.

References

[1] Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum 2, 79 (2018). https://doi.org/10.22331/q-2018-08-06-79

[2] Kandala, A.,Mezzacapo, A., Temme, K., Takita,M., Brink,M., Chow, J.M., Gambetta, J.M.: Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. Nature 549, 242–246 (2017). https://doi.org/10.1038/nature23879

[3] Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm (2014). arXiv:1411.4028

[4] Seo, Y., Heo, J. Edge-based quantum approximate optimization algorithm for MAX-CUT problem. Quantum Inf Process 24, 312 (2025). https://doi.org/10.1007/s11128-025-04925-0